

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv('patient1.csv')
print(df.columns)
```

```
Index(['id', 'sex', 'birth_year', 'country', 'region', 'group',
       'infection_reason', 'infection_order', 'infected_by', 'contact_number',
       'confirmed_date', 'released_date', 'deceased_date', 'state'],
      dtype='object')
```

```
df =pd.read_csv("patient1.csv")
print(df.head())
print(df.info())
```

	id	sex	birth_year	country	region	group	infection_reason	\
0	1	female	1984.0	China	filtered at airport	NaN	visit to Wuhan	
1	2	male	1964.0	Korea	filtered at airport	NaN	visit to Wuhan	
2	3	male	1966.0	Korea	capital area	NaN	visit to Wuhan	
3	4	male	1964.0	Korea	capital area	NaN	visit to Wuhan	
4	5	male	1987.0	Korea	capital area	NaN	visit to Wuhan	

	infection_order	infected_by	contact_number	confirmed_date	released_date	\
0	1.0	NaN	45.0	2020-01-20	2020-02-06	
1	1.0	NaN	75.0	2020-01-24	2020-02-05	
2	1.0	NaN	16.0	2020-01-26	2020-02-12	
3	1.0	NaN	95.0	2020-01-27	2020-02-09	
4	1.0	NaN	31.0	2020-01-30	NaN	

	deceased_date	state
0	NaN	released
1	NaN	released
2	NaN	released
3	NaN	released
4	NaN	isolated

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 4212 entries, 0 to 4211
```

```
Data columns (total 14 columns):
```

#	Column	Non-Null Count	Dtype
0	id	4212 non-null	int64
1	sex	318 non-null	object
2	birth_year	292 non-null	float64
3	country	4212 non-null	object
4	region	305 non-null	object
5	group	76 non-null	object
6	infection_reason	130 non-null	object
7	infection_order	35 non-null	float64
8	infected_by	62 non-null	float64
9	contact_number	32 non-null	float64
10	confirmed_date	4212 non-null	object
11	released_date	28 non-null	object
12	deceased_date	13 non-null	object
13	state	4212 non-null	object

```
dtypes: float64(4), int64(1), object(9)
```

```
memory usage: 460.8+ KB
```

```
None
```

```
date_cols = ['confirmed_date', 'released_date', 'deceased_date']

for col in date_cols:
    df[col] = pd.to_datetime(df[col], errors='coerce')

# Create age
df['age'] = 2020 - df['birth_year']

# Recovery time (in days)
df['recovery_days'] = (df['released_date'] - df['confirmed_date']).dt.days

# Clean unrealistic values
df = df[(df['age'] > 0) & (df['age'] < 110)]
```

```
df = df.dropna(subset=['birth_year']) # important
df['age'] = 2020 - df['birth_year']
```

```
df['recovery_days'] = (df['released_date'] - df['confirmed_date']).dt.days
```

```
df = df.dropna(subset=['recovery_days'])

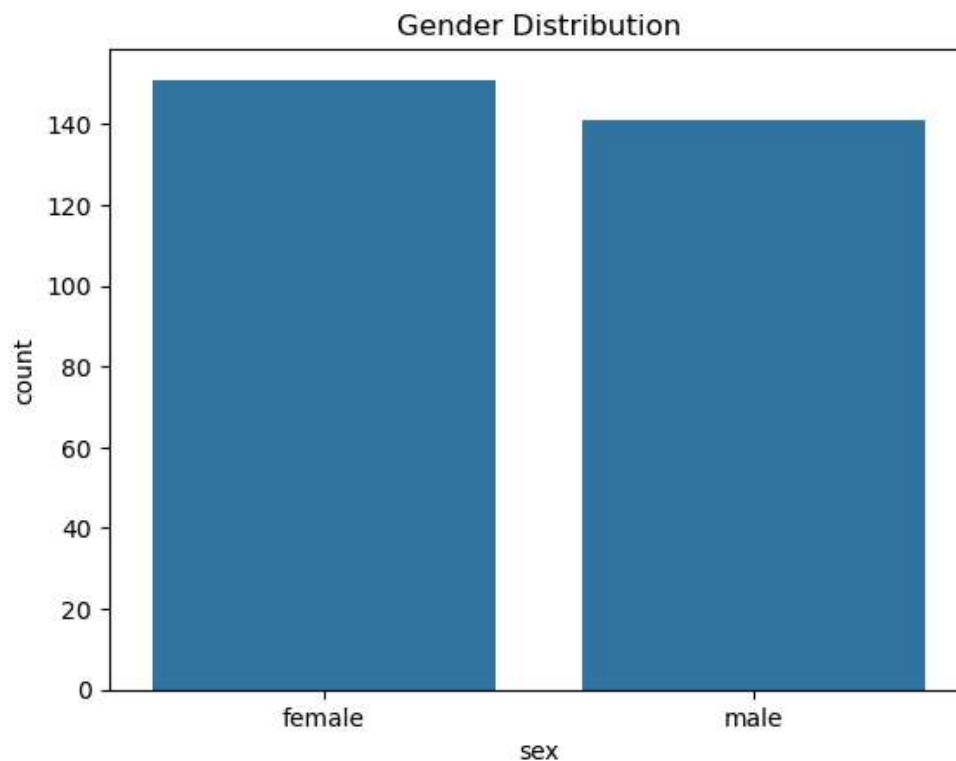
df = df[(df['age'] > 0) & (df['age'] < 110)]

print(df.shape)
```

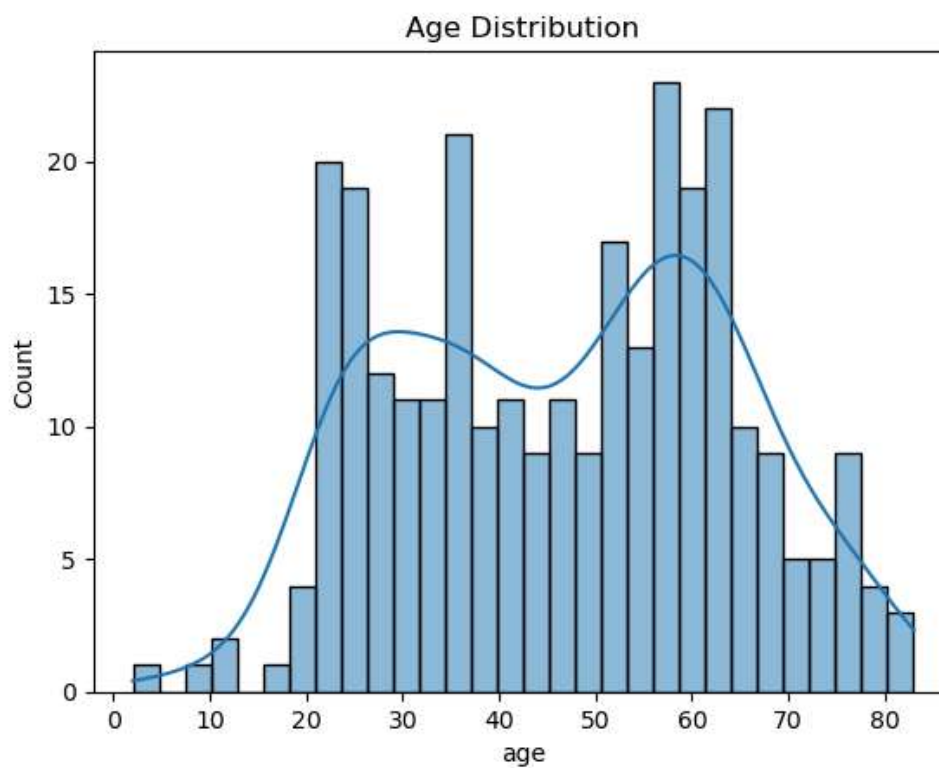
```
(28, 16)
```

```
X = df[["age"]] # ✅ only age
y = df["recovery_days"]
```

```
plt.figure()
sns.countplot(data=df, x='sex')
plt.title("Gender Distribution")
plt.show()
```

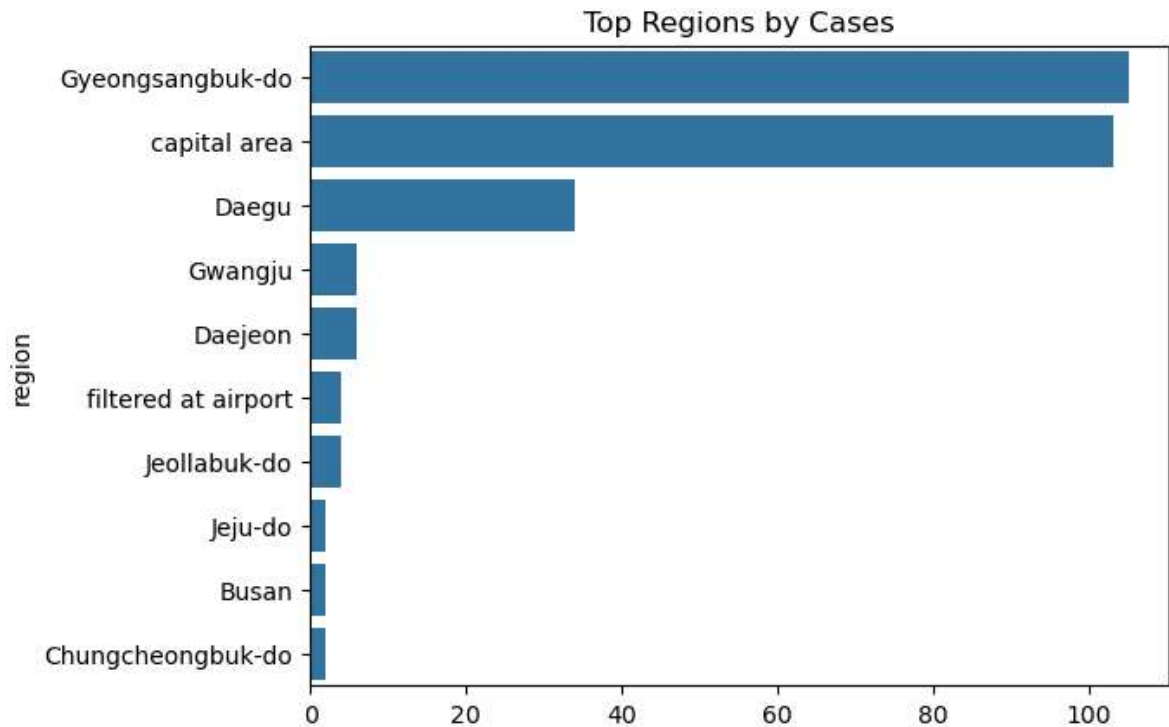


```
plt.figure()  
sns.histplot(df['age'], bins=30, kde=True)  
plt.title("Age Distribution")  
plt.show()
```

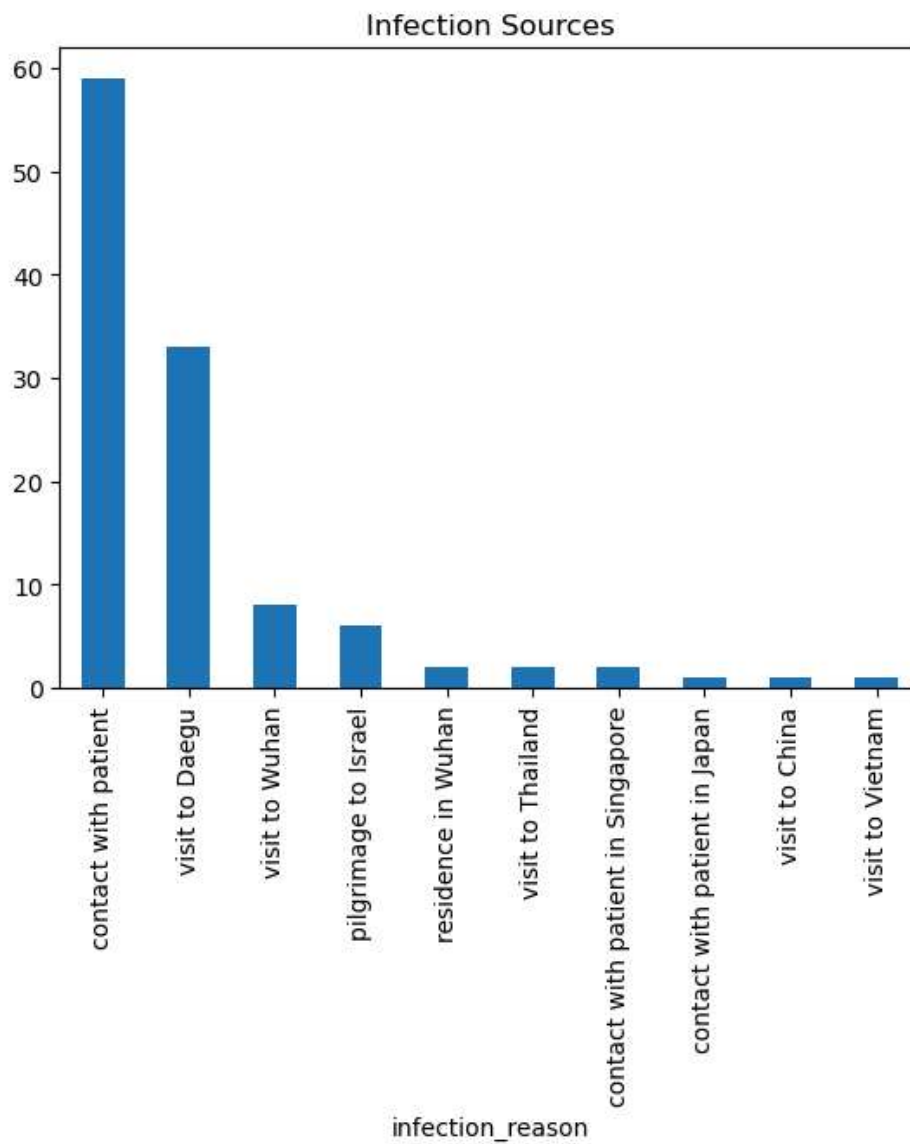


```
top_regions = df['region'].value_counts().head(10)  
  
plt.figure()
```

```
sns.barplot(x=top_regions.values, y=top_regions.index)
plt.title("Top Regions by Cases")
plt.show()
```

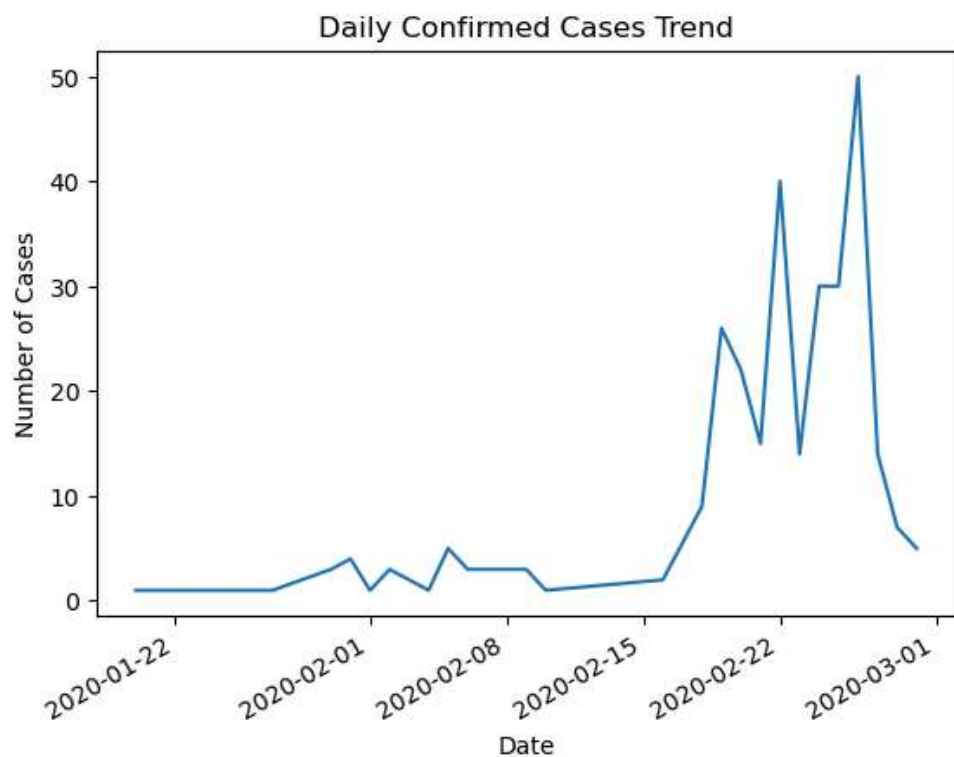


```
plt.figure()
df['infection_reason'].value_counts().head(10).plot(kind='bar')
plt.title("Infection Sources")
plt.show()
```



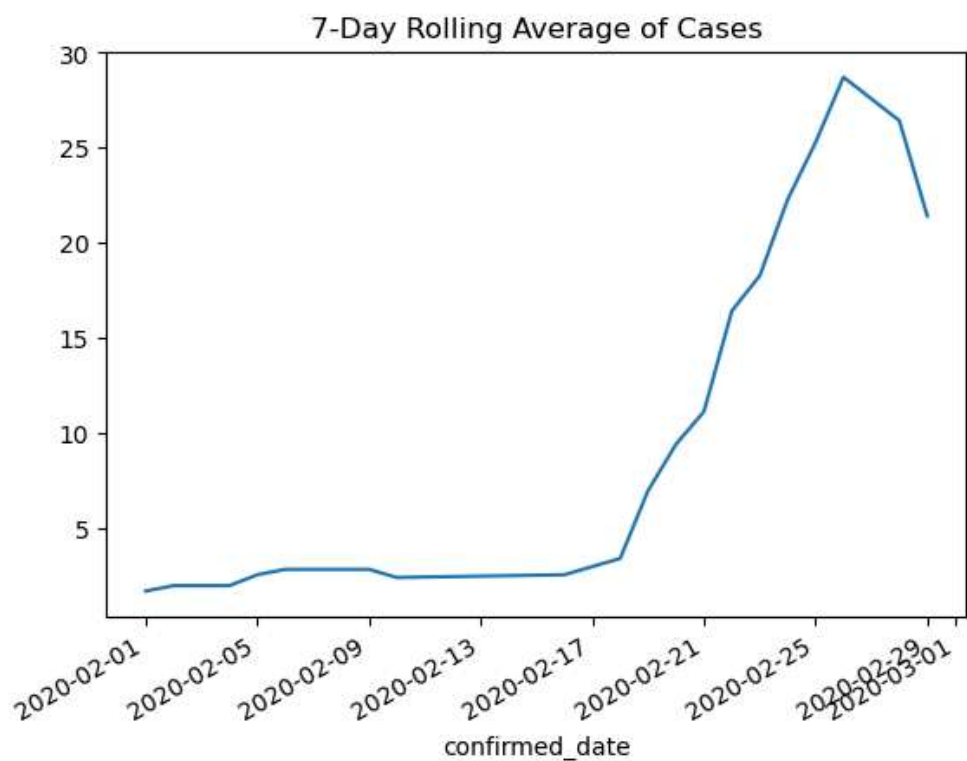
```
daily_cases = df.groupby('confirmed_date').size()
```

```
plt.figure()  
daily_cases.plot()  
plt.title("Daily Confirmed Cases Trend")  
plt.xlabel("Date")  
plt.ylabel("Number of Cases")  
plt.show()
```

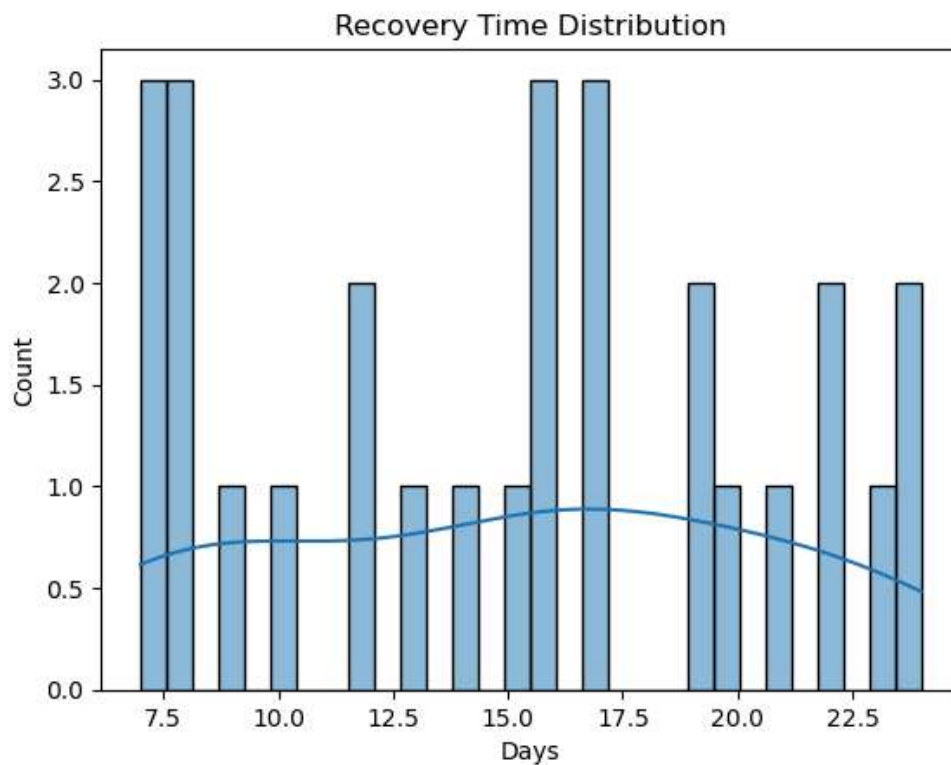


```
rolling_cases = daily_cases.rolling(window=7).mean()
```

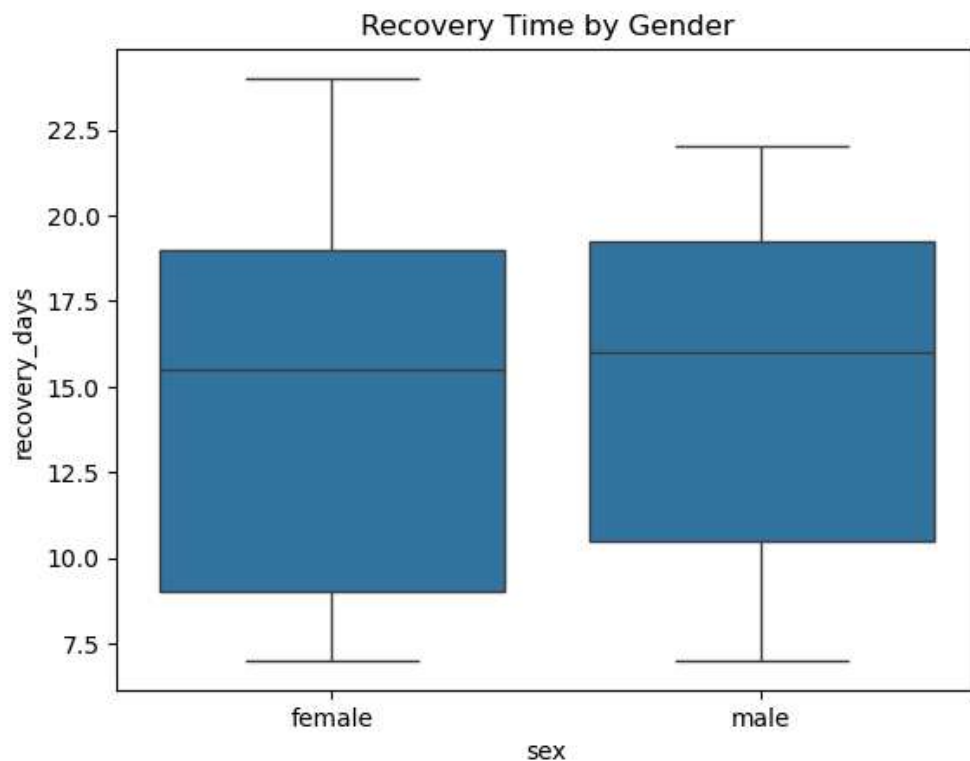
```
plt.figure()  
rolling_cases.plot()  
plt.title("7-Day Rolling Average of Cases")  
plt.show()
```



```
plt.figure()
sns.histplot(df['recovery_days'].dropna(), bins=30, kde=True)
plt.title("Recovery Time Distribution")
plt.xlabel("Days")
plt.show()
```

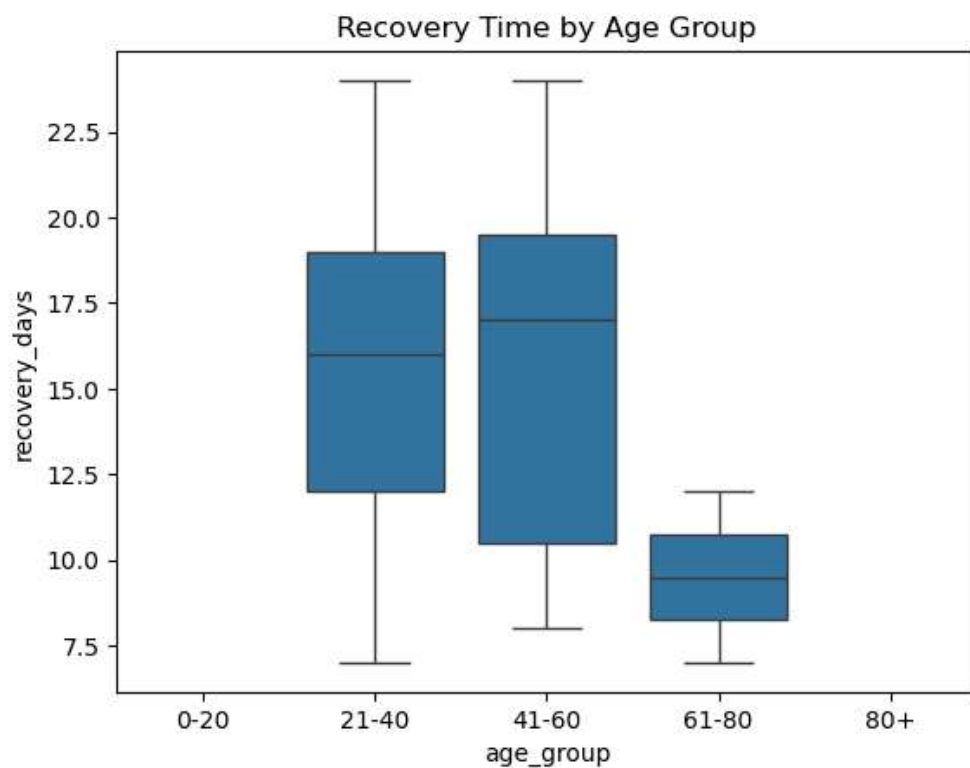


```
plt.figure()
sns.boxplot(data=df, x='sex', y='recovery_days')
plt.title("Recovery Time by Gender")
plt.show()
```



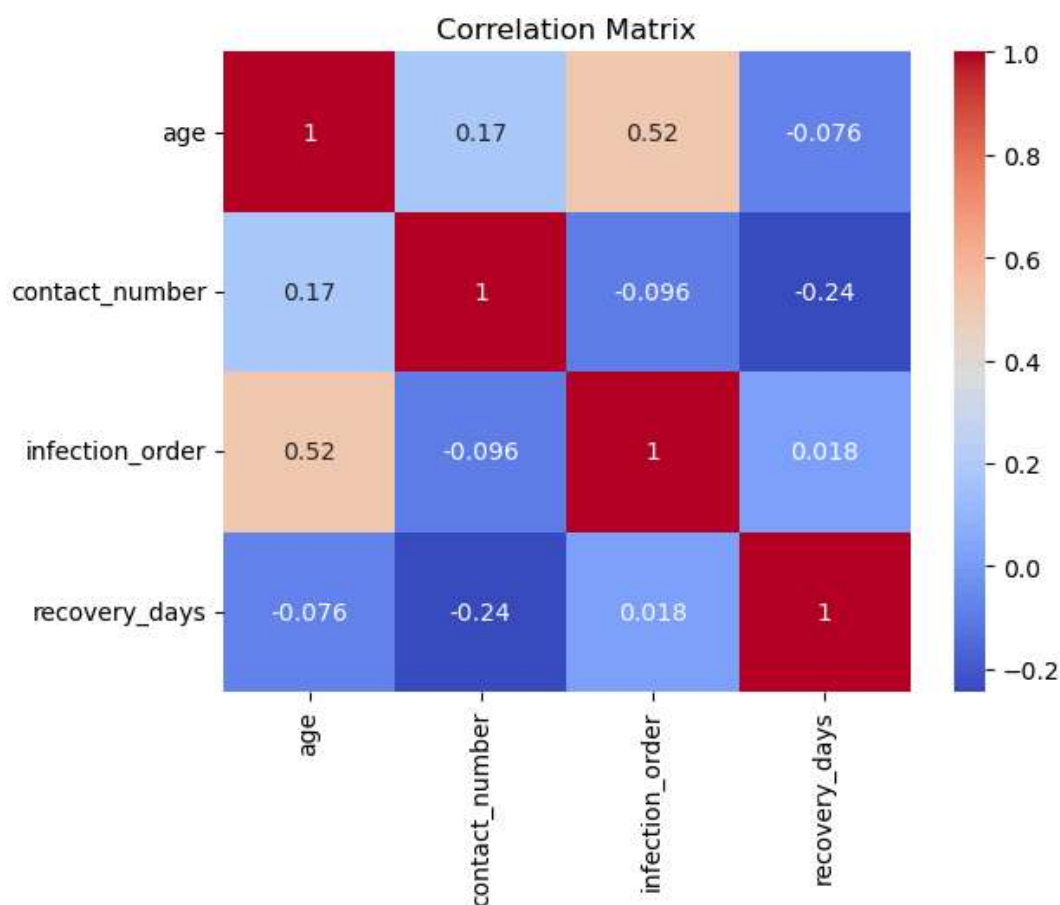
```
df['age_group'] = pd.cut(df['age'], bins=[0,20,40,60,80,100],  
                          labels=['0-20', '21-40', '41-60', '61-80', '80+'])
```

```
plt.figure()  
sns.boxplot(data=df, x='age_group', y='recovery_days')  
plt.title("Recovery Time by Age Group")  
plt.show()
```




```
corr_df = df[['age', 'contact_number', 'infection_order', 'recovery_days']]
```

```
plt.figure()
sns.heatmap(corr_df.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
```



```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Prepare data
model_df = df[['age', 'contact_number', 'infection_order', 'recovery_days']].dropna()

X = model_df[['age', 'contact_number', 'infection_order']]
y = model_df['recovery_days']

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Evaluate
print("Model Score:", model.score(X_test, y_test))
print("Coefficients:", model.coef_)
```

```
Model Score: -7.7876771557266125
Coefficients: [ 0.15024098 -0.05828484 -0.44235362]
```

```
df.groupby('age')
['recovery_days']
```

```
['recovery_days']
```

```
df.groupby('age')['recovery_days'].mean()
```

```
age
2.0    NaN
9.0    NaN
11.0   NaN
16.0   NaN
19.0   NaN
..
77.0   NaN
78.0   NaN
79.0   NaN
82.0   NaN
83.0   NaN
Name: recovery_days, Length: 67, dtype: float64
```

```
df[['contact_number', 'recovery_days']].corr()
```

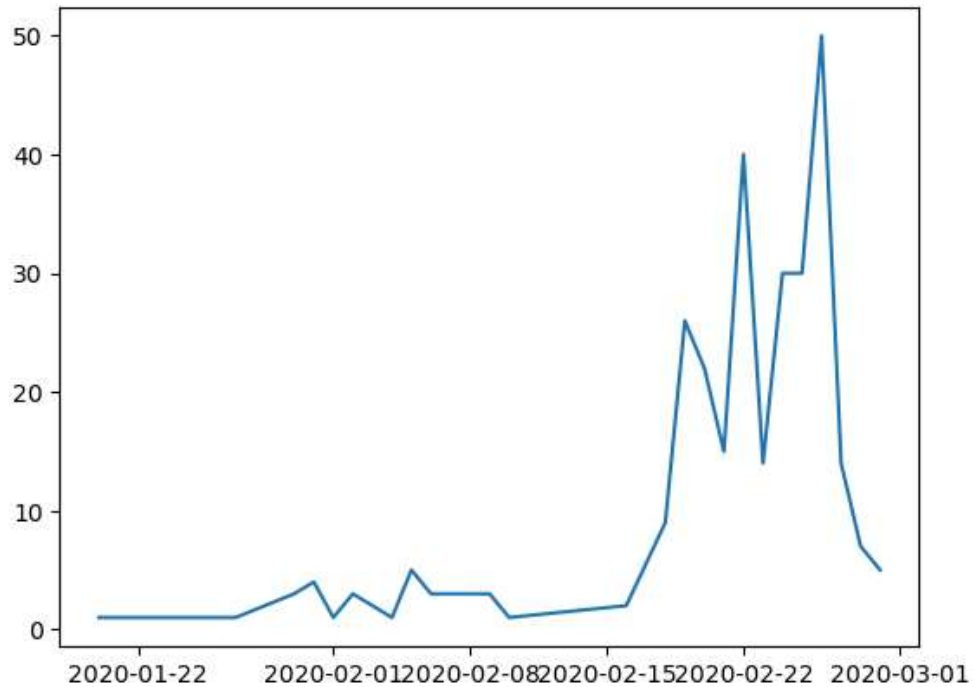
	contact_number	recovery_days
contact_number	1.000000	-0.244144
recovery_days	-0.244144	1.000000

```
df.groupby('confirmed_date').size()
```

```
confirmed_date
2020-01-20    1
2020-01-24    1
2020-01-26    1
2020-01-27    1
2020-01-30    3
2020-01-31    4
2020-02-01    1
2020-02-02    3
2020-02-04    1
2020-02-05    5
2020-02-06    3
2020-02-09    3
2020-02-10    1
2020-02-16    2
2020-02-18    9
2020-02-19   26
2020-02-20   22
2020-02-21   15
2020-02-22   40
2020-02-23   14
2020-02-24   30
2020-02-25   30
2020-02-26   50
2020-02-27   14
2020-02-28    7
2020-02-29    5
dtype: int64
```

```
plt.plot(df.groupby('confirmed_date').size())
```

```
[<matplotlib.lines.Line2D at 0x2792fa91f90>]
```



```
df['region'].value_counts()
```

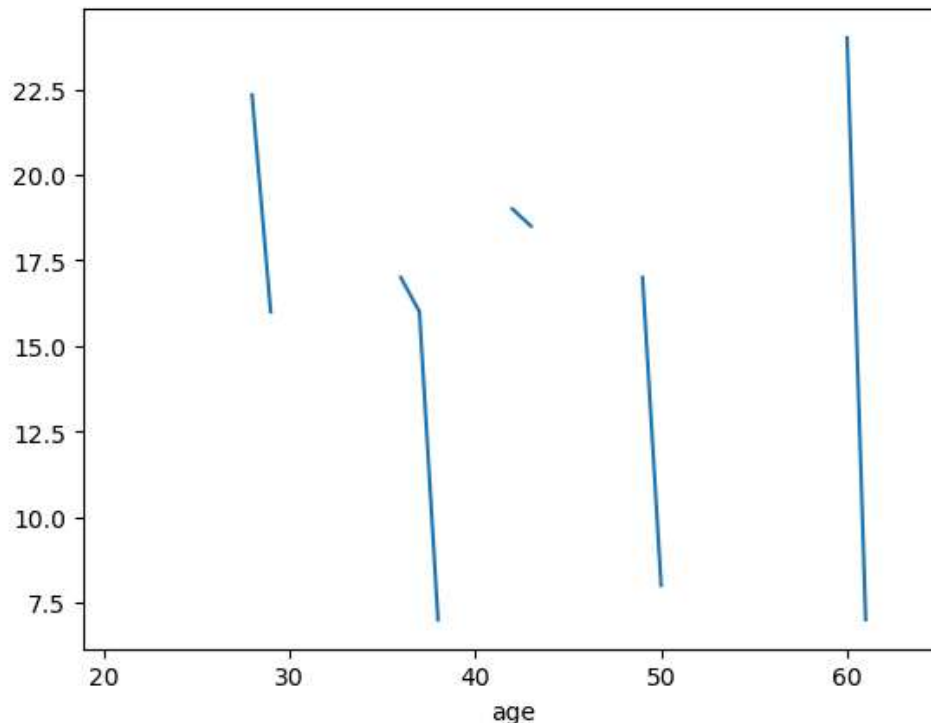
```
region
Gyeongsangbuk-do    105
capital area        103
Daegu                34
Gwangju              6
Daejeon              6
filtered at airport  4
Jeollabuk-do         4
Jeju-do              2
Busan                2
Chungcheongbuk-do    2
Ulsan                2
Gangwon-do           1
Chungcheongnam-do    1
Name: count, dtype: int64
```

Start coding or [generate](#) with AI.

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: streamlit in c:\programdata\anaconda3\lib\site-packages (1.51.0)
Requirement already satisfied: altair!=5.4.0,!5.4.1,<6,>=4.0 in c:\programdata\anaconda3\lib\site-packages (4.2.0)
Requirement already satisfied: blinker<2,>=1.5.0 in c:\programdata\anaconda3\lib\site-packages (1.5.0)
Requirement already satisfied: cachetools<7,>=4.0 in c:\programdata\anaconda3\lib\site-packages (4.2.1)
Requirement already satisfied: click<9,>=7.0 in c:\programdata\anaconda3\lib\site-packages (8.0.3)
Requirement already satisfied: numpy<3,>=1.23 in c:\programdata\anaconda3\lib\site-packages (1.23.0)
Requirement already satisfied: packaging<26,>=20 in c:\programdata\anaconda3\lib\site-packages (23.0)
Requirement already satisfied: pandas<3,>=1.4.0 in c:\programdata\anaconda3\lib\site-packages (1.4.0)
Requirement already satisfied: pillow<13,>=7.1.0 in c:\programdata\anaconda3\lib\site-packages (9.0.1)
Requirement already satisfied: protobuf<7,>=3.20 in c:\programdata\anaconda3\lib\site-packages (3.20.3)
Requirement already satisfied: pyarrow<22,>=7.0 in c:\programdata\anaconda3\lib\site-packages (11.0.0)
Requirement already satisfied: requests<3,>=2.27 in c:\programdata\anaconda3\lib\site-packages (2.27.1)
Requirement already satisfied: tenacity<10,>=8.1.0 in c:\programdata\anaconda3\lib\site-packages (8.1.0)
Requirement already satisfied: toml<2,>=0.10.1 in c:\programdata\anaconda3\lib\site-packages (0.10.1)
```

Requirement already satisfied: typing-extensions<5,>=4.4.0 in c:\programdata\anaconda3\lib\site-packages
 Requirement already satisfied: watchdog<7,>=2.1.5 in c:\programdata\anaconda3\lib\site-packages
 Requirement already satisfied: gitpython!=3.1.19,<4,>=3.0.7 in c:\programdata\anaconda3\lib\site-packages
 Requirement already satisfied: tornado!=6.5.0,<7,>=6.0.3 in c:\programdata\anaconda3\lib\site-packages
 Requirement already satisfied: jinja2 in c:\programdata\anaconda3\lib\site-packages (from altai
 Requirement already satisfied: jsonschema>=3.0 in c:\programdata\anaconda3\lib\site-packages (f
 Requirement already satisfied: narwhals>=1.14.2 in c:\programdata\anaconda3\lib\site-packages (f
 Requirement already satisfied: colorama in c:\programdata\anaconda3\lib\site-packages (from cli
 Requirement already satisfied: gitdb<5,>=4.0.1 in c:\programdata\anaconda3\lib\site-packages (f
 Requirement already satisfied: smmap<6,>=3.0.1 in c:\programdata\anaconda3\lib\site-packages (f
 Requirement already satisfied: python-dateutil>=2.8.2 in c:\programdata\anaconda3\lib\site-pack
 Requirement already satisfied: pytz>=2020.1 in c:\programdata\anaconda3\lib\site-packages (from
 Requirement already satisfied: tzdata>=2022.7 in c:\programdata\anaconda3\lib\site-packages (fr
 Requirement already satisfied: charset_normalizer<4,>=2 in c:\programdata\anaconda3\lib\site-pa
 Requirement already satisfied: idna<4,>=2.5 in c:\programdata\anaconda3\lib\site-packages (from
 Requirement already satisfied: urllib3<3,>=1.21.1 in c:\programdata\anaconda3\lib\site-packages
 Requirement already satisfied: certifi>=2017.4.17 in c:\programdata\anaconda3\lib\site-packages
 Requirement already satisfied: attrs>=22.2.0 in c:\programdata\anaconda3\lib\site-packages (fro
 Requirement already satisfied: jsonschema-specifications>=2023.03.6 in c:\programdata\anaconda3
 Requirement already satisfied: referencing>=0.28.4 in c:\programdata\anaconda3\lib\site-package
 Requirement already satisfied: rpds-py>=0.7.1 in c:\programdata\anaconda3\lib\site-packages (fr
 Requirement already satisfied: six>=1.5 in c:\programdata\anaconda3\lib\site-packages (from pyt
 Requirement already satisfied: MarkupSafe>=2.0 in c:\programdata\anaconda3\lib\site-packages (f
 Note: you may need to restart the kernel to use updated packages.

```
df.groupby('age')['recovery_days'].mean().plot()
plt.show()
```



```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=4
)
```

```

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("R2 Score:", r2_score(y_test, y_pred))

```

R2 Score: 0.0050868901027785185

```

import pandas as pd

# Load your file
df = pd.read_csv("patient1.csv")

# Check first few rows
print(df.head())

# Check columns
print(df.columns)

# Check shape (rows, columns)
print(df.shape)

```

```

      id  sex  birth_year  country  region  group  infection_reason \
0     1  female    1984.0    China  filtered at airport  NaN  visit to Wuhan
1     2   male    1964.0    Korea  filtered at airport  NaN  visit to Wuhan
2     3   male    1966.0    Korea    capital area  NaN  visit to Wuhan
3     4   male    1964.0    Korea    capital area  NaN  visit to Wuhan
4     5   male    1987.0    Korea    capital area  NaN  visit to Wuhan

      infection_order  infected_by  contact_number  confirmed_date  released_date \
0                   1.0         NaN           45.0    2020-01-20    2020-02-06
1                   1.0         NaN           75.0    2020-01-24    2020-02-05
2                   1.0         NaN           16.0    2020-01-26    2020-02-12
3                   1.0         NaN           95.0    2020-01-27    2020-02-09
4                   1.0         NaN           31.0    2020-01-30         NaN

      deceased_date  state
0                NaN  released
1                NaN  released
2                NaN  released
3                NaN  released
4                NaN  isolated
Index(['id', 'sex', 'birth_year', 'country', 'region', 'group',
      'infection_reason', 'infection_order', 'infected_by', 'contact_number',
      'confirmed_date', 'released_date', 'deceased_date', 'state'],
      dtype='object')
(4212, 14)

```

```
print(df.shape)
```

(4212, 14)

```

df['confirmed_date'] = pd.to_datetime(df['confirmed_date'])
df['released_date'] = pd.to_datetime(df['released_date'])

# Create age (from birth_year)
df['age'] = 2020 - df['birth_year']

# Create recovery days

```

```
df['recovery_days'] = (df['released_date'] - df['confirmed_date']).dt.days

# Remove missing values
df = df.dropna(subset=['age', 'recovery_days'])

# Check
print(df[['age', 'recovery_days']].head())
```

	age	recovery_days
0	36.0	17.0
1	56.0	12.0